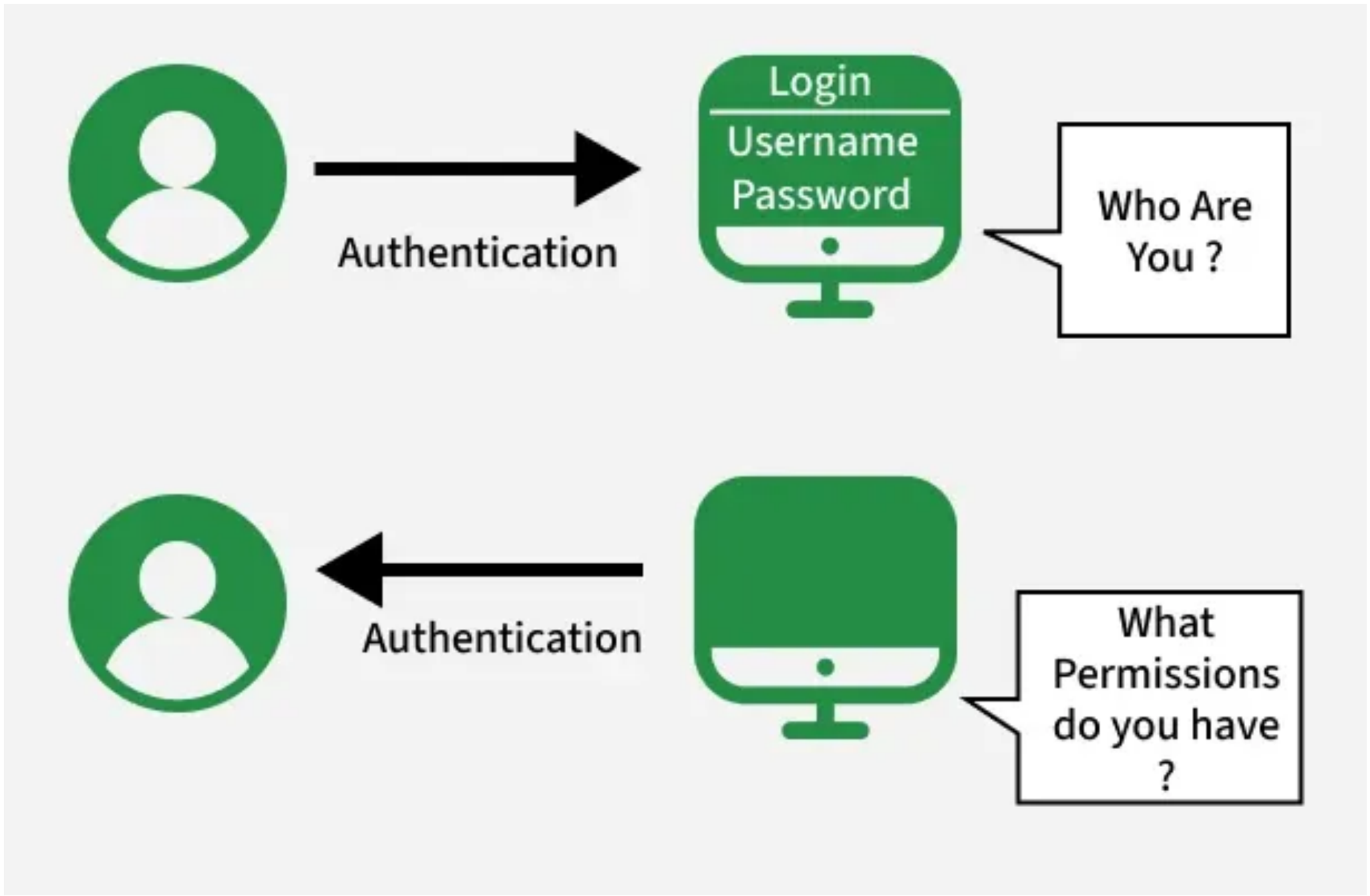
Auth

인증(Authentication vs Authorization)



해쉬함수

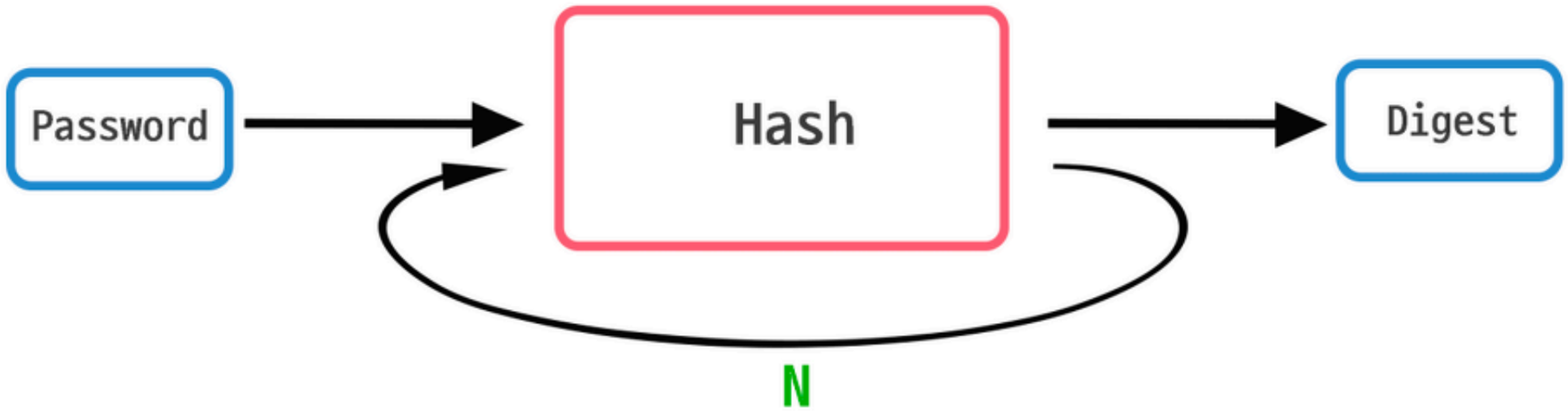
해시 함수는 임의 길이의 데이터를 고정 길이의 해시값으로 변환하는 함수

https://st-lab.tistory.com/100

해쉬알고리즘

알고리즘	출력 길이	특징	사용여부
MD5	128bit (16바이트)	매우 빠름	x
SHA-1	160bit (20바이트)	과거 표준	x
SHA-2 (SHA-256)	256bit (32바이트)	현재 가장 널리 사용	O
SHA-2 (SHA-512)	512bit (64바이트)	SHA-256보다 긴 출력	O
SHA-3	224~512bit	SHA-2의 후속 표준	O

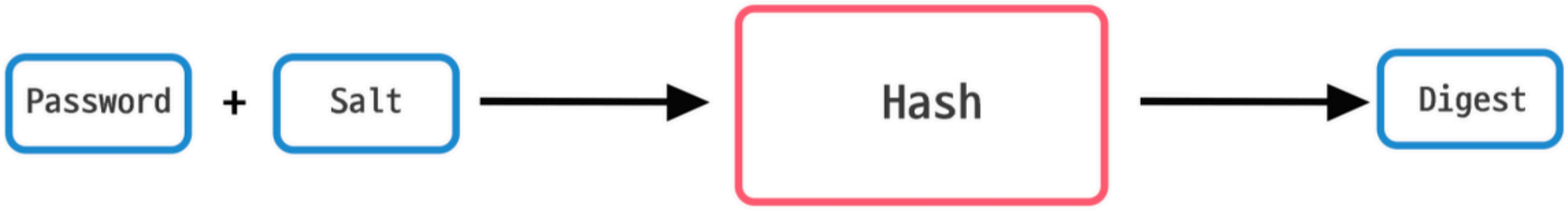
Key Stretching



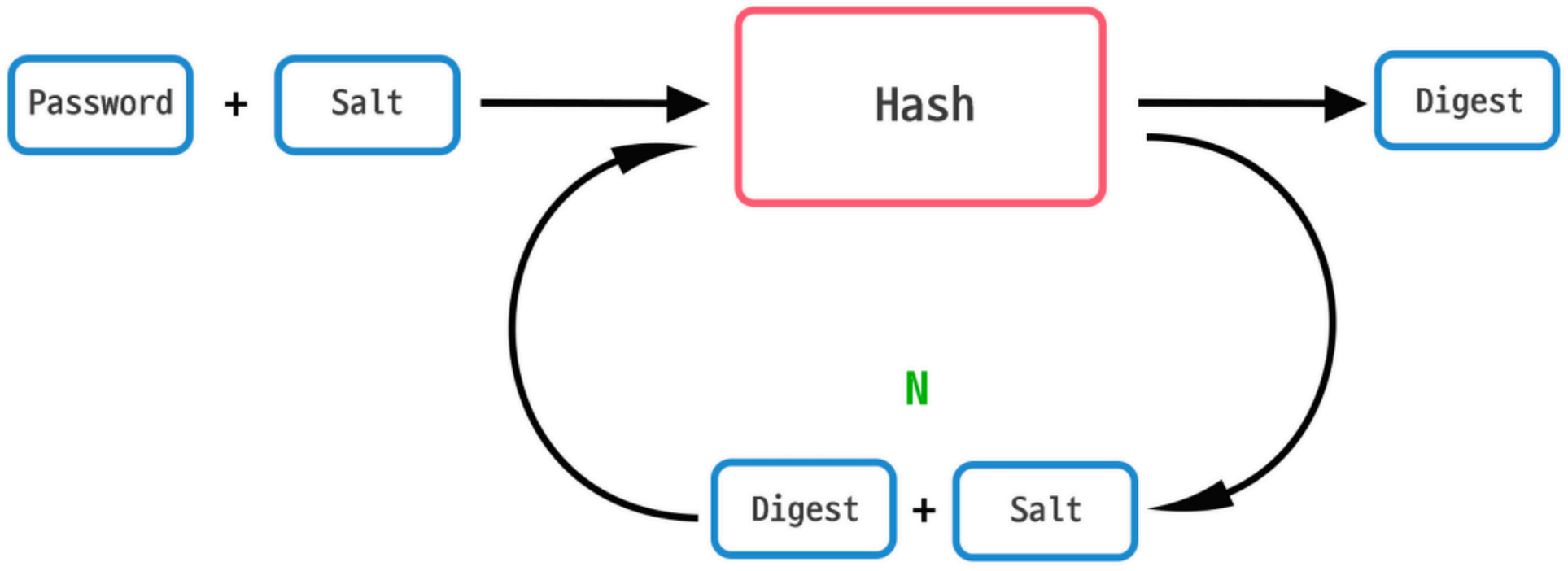
Hash 함수를 여러 번(Cost Factor) 실행

https://st-lab.tistory.com/100

Password Hashing



Hash 함수를 실행하기 이전에 임의의 문자열(salt)을 덧붙임



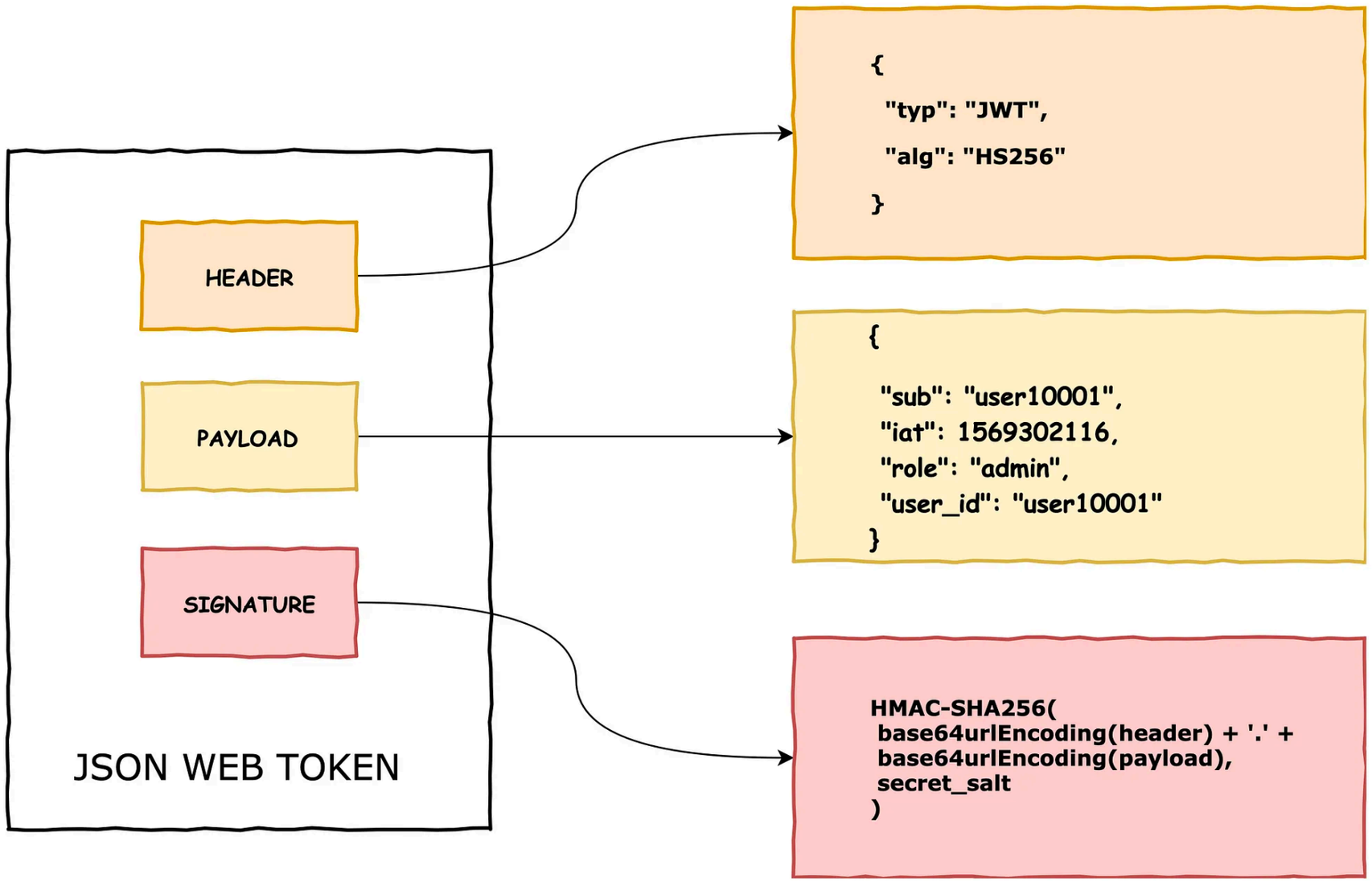
Hash 함수를 여러번 실행하기 이전에 임의의 문자열(salt)을 덧붙임

https://st-lab.tistory.com/100

bcrypt 사용법

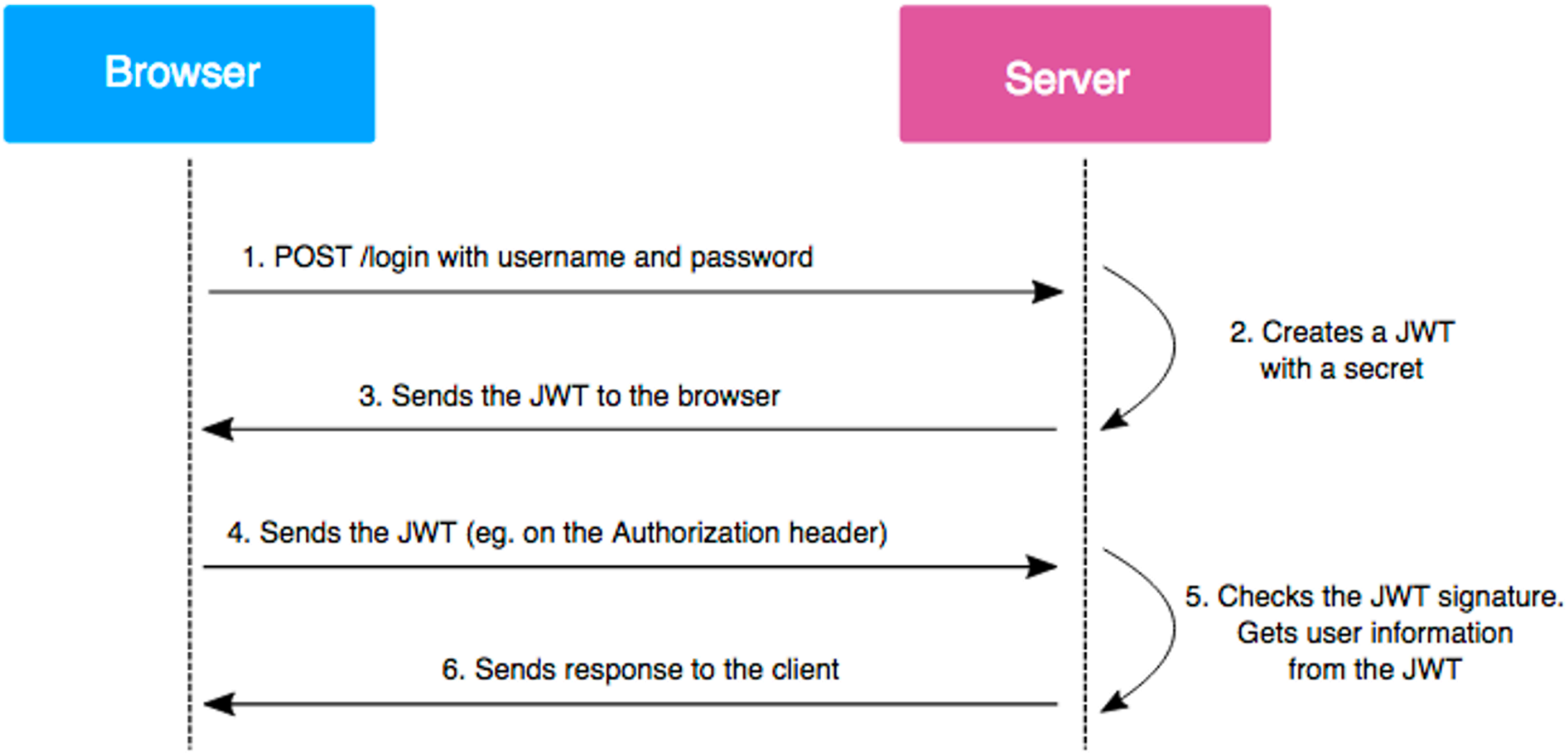
```
const bcrypt = require('bcrypt');
const password = 'Pass1234';
const saltRound = 10;
//sync
let hashed = bcrypt.hashSync(password, saltRound);
console.log(`password: ${password}, hashed:${hashed}`);
const result = bcrypt.compareSync(password, hashed);
console.log(`Password is Same: ${result}`);

//async
(async () => {
  const hashed = await bcrypt.hash(password, saltRound);
  console.log(`password: ${password}, hashed:${hashed}`);
  const result = await bcrypt.compare(password, hashed);
  console.log(`Password is Same: ${result}`);
})();
```

JWT(Json Web Token)



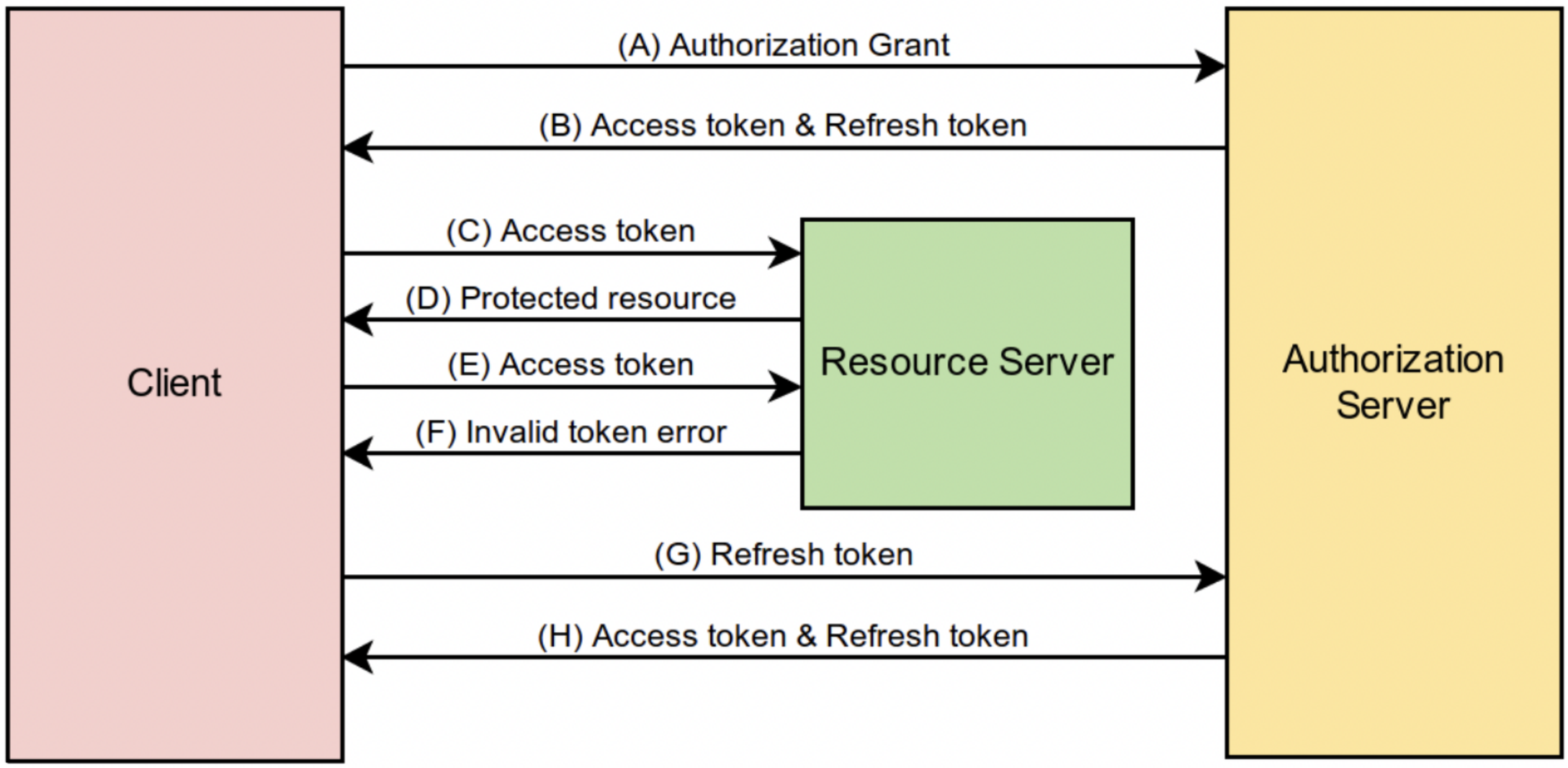
https://cdn.hashnode.com

JWT(Json Web Token)



https://www.vaadata.com

Access, Refresh Token



https://velog.velcdn.com/images/gkdbssla97/post/539f04af-ff78-4111-be1a-0bcfaa592887/image.png

jwt 사용법

```
const jwt = require('jsonwebtoken');
const secret = 'wf3K]s04C7T3'; // .env에 숨김
const token = jwt.sign({ id: 1, name: '김철수', lvl: 3 }, secret, {
  expiresIn: '1h',
});

console.log('토큰:', token);
const payload = jwt.verify(token, secret);
console.log('페이로드 : ', payload);

const token1 = 'eyJhbGciOiJIUz.....';
try {
  jwt.verify(token1, secret);
} catch (err) {
  console.log('위변조 토큰 거부 : ', err.message);
}
```

Password Hashing

```
app.post('/register', async (req, res) => {
  const { name, email, password } = req.body;
  if (!name || !email || !password) {
    return res
      .status(400)
      .json({ success: false, message: 'name, email, password는 필수입니다.' });
  }
  if (users.find((user) => user.email === email)) {
    return res
      .status(409)
      .json({ success: false, message: '이미 가입한 이메일 입니다.' });
  }
  const hash = await bcrypt.hash(password, saltRound);
  const user = { id: userId++, name, email, password: hash };
  users.push(user);
  res.status(201).json({
    success: true,
    data: { id: user.id, name: user.name, email: user.email },
  });
});
```

jwt 발행

```
app.post('/login', async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password) {
    return res
      .status(400)
      .json({ success: false, message: 'email, password는 필수입니다.' });
  }
  const user = users.find((user) => user.email === email);
  if (!user || !(await bcrypt.compare(password, user.password))) {
    return res.status(401).json({
      success: false,
      message: 'email 혹은 password가 올바르지 않습니다.',
    });
  }
  const token = jwt.sign({ id: user.id, name: user.name }, secret, {
    expiresIn: '1h',
  });
  res.json({ success: true, token });
});
```

인증 미들웨어

```
function auth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;

  if (!token) {
    return res
      .status(401)
      .json({ success: false, message: '토큰이 없습니다.' });
  }
  try {
    req.user = jwt.verify(token, secret);
    next();
  } catch (error) {
    res
      .status(401)
      .json({ success: false, message: '유효하지 않은 토큰입니다.' });
  }
}
```

인증 사용 예

```
app.post('/posts', auth, (req, res) => {
  const { title, content } = req.body;
  if (!title || !content) {
    return res
      .status(400)
      .json({ success: false, message: 'title, content는 필수입니다.' });
  }
  const post = { id: postId++, title, content, name: req.user.name };
  posts.push(post);
  res.status(201).json({ success: true, post });
});

app.get('/posts', auth, (req, res) => {
  res.status(200).json({ success: true, posts });
});
```